

The Heteroclinic Cap: Itinerant CAROM Dynamics over a Frozen Language Model

Working note — companion to the CAROM semantics and sandbox reports

July 2026

Abstract

A *heteroclinic cap* is a thin, trainable dynamical-systems layer placed on top of a frozen pretrained transformer. The transformer interprets a natural-language task description — what the subtasks are, and how they depend on one another — and its representations parameterize a content-addressable operator machine (CAROM) whose control flow is realized as a stable heteroclinic channel: a chain of metastable states through which the computation travels, one subtask per station. The division of labor is the design thesis: the language model is the *compiler* (semantics, dependency structure, zero-shot generalization over phrasings); the itinerant dynamics are the *runtime* (persistent gated state, phase discipline, ordered execution, learned halting, and an interpretable execution trace). We give the reasoning behind the construction, its predicted strengths and failure modes, the results of a CPU sandbox implementation — which validated the mechanism end-to-end and isolated the base model’s semantic quality as the single limiting factor — and the next phase of GPU-scale experiments, including a second experiment in which the channel topology itself is compiled per-instance from the language model’s reading of dependency phrases.

1 The idea and the reasoning behind it

1.1 Two components, each supplying what the other lacks

Standalone CAROM — a library of K reusable read–transform–gate–write operators over a slot workspace, routed by commands, executed as an itinerant flow through a heteroclinic channel — was shown in earlier notes to be mechanically sound. Its sandbox evaluation also exposed its two structural poverties:

- (P1) **Opaque commands.** Routing by embedding lookup means command semantics are learned per-token from task reward. There is no generalization to unseen commands, and the realizable instruction set is bounded by K (the routing-bottleneck constraint), forcing carefully hand-designed command vocabularies.
- (P2) **A single global ordering prior.** Sequential structure lives in one learned inhibition matrix ρ shared across all programs. Discovering per-task execution order from reward alone (“free- ρ ”) was empirically the hard open problem: the architecture must learn *one* prior where the truth is per-instance.

A frozen pretrained language model dissolves both, because both are *language-understanding* problems wearing an architecture costume. Commands stop being tokens and become LLM-interpreted spans: the routing mixture becomes a hypernetwork $\rho_k = g(h_m)$ on the LLM

representation h_m of subtask m , so a paraphrased or never-seen instruction lands near its semantic neighbors and inherits their operator mixture — zero-shot routing that a lookup table structurally cannot provide. And the inhibition matrix becomes per-instance and compiled: $\rho_{mj} = f(h_m, h_j)$, a pairwise scorer over subtask-pair representations, reads “after the list is reversed, double every value” as a dependency edge and writes it into the channel topology. Slogan form: **the LLM is the compiler; the heteroclinic channel is the runtime.**

Conversely, the dynamics supply what autoregressive LLM execution lacks. Chain-of-thought has no persistent state discipline: intermediate results live in the token stream, subject to drift, self-overwriting, recency decay, and no per-step completion guarantee; empirically, LLMs lose track in long state-manipulation chains whose individual steps are trivial. The cap provides exactly the missing invariants: a register workspace with gated writes (state cannot silently decay); phase structure from the channel (subtask m *completes* — the workspace relaxes under its constraint — before $m+1$ gains control); within-phase equilibrium semantics (local error correction); an absorbing halt mode (learned, variable-length termination); and a control trajectory $a(t)$ that constitutes a readable trace of what was executed, when, and for how long.

1.2 Predicted strengths

- (S1) **Zero-shot instruction generalization.** Routing inherits the base model’s semantic geometry: new phrasings of known operations route correctly without cap retraining. The routing bottleneck also relaxes, because command identity is no longer the information channel — meaning is.
- (S2) **Per-instance control flow.** Compiled ρ turns the free- ρ emergence problem into supervised (or distilled) edge prediction: the LLM can annotate its own dependency analyses, converting order discovery from pure emergence into learning with an abundant, imperfect teacher, corrected by end-task gradients.
- (S3) **Execution-length robustness.** The target regime is tasks bounded by state fidelity and sequence length rather than per-step intelligence. The cap’s accuracy should degrade more slowly with program length than the base model’s chain-of-thought.
- (S4) **Interpretability by construction.** The itinerary (stations visited, dwell times), the routing mixtures, and the workspace snapshots form a three-level trace: which subtask ran, which primitives implemented it, and what it did to the state.
- (S5) **A learning-efficiency bonus (found, not predicted).** Routing through shared representations couples parameters across similar commands; independent token embeddings do not. The sandbox found this doubles seen-command learning at matched steps — the cap pays for itself before the transfer question is even posed.
- (S6) **An arbitration reading.** In the agentic variant, operators or phases invoke the LLM on subtasks, and the GLV competition becomes an arbitration core over agent invocations — activation-economics-flavored control with an explicit dynamical semantics, pointing toward multi-agent orchestration uses beyond single-model capping.

1.3 Predicted weaknesses and failure modes

Adapter mismatch (large, anisotropic LLM states read through thin projections; layer choice matters; procedural content may be less linearly accessible than declarative content); the re-

dundancy trap (on tasks the base model solves alone in tokens, the cap is overhead — the case must be made where the baseline demonstrably fails); the distillation ceiling (if the edge scorer only learns dependencies the LLM already articulates, the cap’s contribution is discipline rather than knowledge — acceptable, but it must be claimed as such); and inherited channel engineering (the saddle tuning package from the standalone work: entry conditions, activation floors, fatigue, passage-time clocks).

One further design hypothesis, due to the project’s originator: shaping the base model’s representations with an error-predictive-coding (EPC) objective before freezing may reduce adapter mismatch, since predictive-coding training favors states that are linearly predictive — precisely the property a thin near-linear cap consumes — and PC inference is itself relaxation to equilibrium, plausibly a better impedance match to a cap whose phases are constraint relaxations. This is formulated as an A/B in the next phase.

2 Sandbox implementation and results

2.1 Setup

CPU-only container; no pretrained checkpoints available, so the frozen “LLM” is a from-scratch stand-in: a 3-layer, $d=96$ causal transformer ($\sim 0.4\text{M}$ parameters) pretrained by next-token prediction on the task’s own text distribution, then frozen. Task: programs of 2–4 commands over seven whole-sequence primitives on $\mathbb{Z}_8$ (increment, double, negate, reverse, rotate, swap, prefix-sum), each rendered in natural language by one of three paraphrase templates; variants 0,1 train the cap, variant 2 is held out (the base pretrains on all three, unsupervised — the honest miniature of “base saw all language, cap trained on a subset”). Command representations: mean of frozen last-layer states over each command’s span. Cap models: CAPSCHEDULED (hypernetwork routing, scheduled execution), TOKSCHEDULED (identical but routed from opaque per-(primitive, variant) token embeddings — structurally blind to unseen paraphrases), and CAPITINERANT (heteroclinic wrapper: LM-derived routing and fitness bias, hand-built successor chain, padded modes suppressed through the fitness channel). Single seed; small budgets; trends only.

2.2 Results

R1: the mechanism works end-to-end. CAPITINERANT trains stably by BPTT through ~ 56 coupled integration steps and executes **192/192 perfectly sequential itineraries at every program length $L \in \{2, 3, 4\}$** , with padded modes pinned at the activation floor — i.e., the channel runs exactly L stations for a length- L program. Variable-length heteroclinic execution over frozen-transformer routing is mechanically sound and robust to training. Accuracy is above chance and depth-graded (0.42/0.31/0.29) at a ~ 370 -step budget.

R2: representation routing beats token routing on seen data. At matched steps (~ 1200), CAPSCHEDULED reached 78.5% on training paraphrases versus 41.7% for TOKSCHEDULED (chance 12.5%). Shared representational structure acts as an inductive bias across commands; independent embeddings learn each token from scratch.

R3: transfer is systematic exactly where representation similarity exists. Held-out-paraphrase accuracy by primitive (cap / token): shr 1.00/0.12; dbl 1.00/1.00*; rev 0.35/0.16;

neg 0.18/0.10; inc 0.11/0.25*; swp 0.14/0.04; cms 0.12/0.15 (*token-baseline successes are attributable to untrained random-init embeddings landing near useful operator corners — luck, not mechanism; hence the multi-seed requirement below). The cap transfers perfectly on primitives whose held-out paraphrase shares content words with the training variants, partially on partial overlap, and at chance on zero overlap.

R4: the base model is the isolated limiting factor. A nearest-centroid probe of the frozen representations identifies held-out paraphrases’ primitives at only 0.27 (chance 0.14), with a per-primitive pattern exactly mirroring lexical overlap. An attempt to induce effect-based (semantic) equivalence by grounding the pretraining corpus with worked input/output examples failed: the tiny model’s loss plateaued without learning to execute, so effect-based representations never formed. Three independent measurements — the probe, the grounding failure, and the end-to-end transfer pattern — triangulate one conclusion: *the cap architecture works; what is missing is a base that knows what the commands mean.*

2.3 What the sandbox shows, and what it does not

Shown (at toy scale, single seed): the full cap pipeline trains; heteroclinic execution over LM-derived routing is stable, sequential, and length-adaptive; representation routing dominates token routing on seen commands; transfer tracks representation similarity with the token control at (noisy) chance; and the limiting factor is cleanly attributed to base semantic quality rather than to the cap.

Not shown: semantic (non-lexical) zero-shot transfer — the central claim — because the stand-in base cannot supply semantic similarity; any comparison against a chain-of-thought baseline (the stand-in cannot do CoT); compiled per-instance channel topology (the sandbox used a fixed chain); the EPC hypothesis; effects at realistic scale; and statistical robustness (everything is $n=1$ seeds). The sandbox’s contribution is that the GPU phase now tests a *single isolated variable* with pre-registered predictions, rather than debugging an apparatus.

3 Next phase: GPU experiments

Experiment 1: semantic transfer (base swap). Identical pipeline, frozen GPT-2-small (124M) as base ($LM_D=768$; span-mean pooling with BPE offset tracking; probe a small sweep of layers). Gate: nearest-centroid probe $\gtrsim 0.8$ before training the cap. Pre-registered prediction: held-out transfer on the zero-overlap primitives (cms, swp, inc) rises from $\approx$ chance toward the seen-paraphrase level; the token control does not move. Three seeds minimum; report per-primitive tables. Secondary: does R2’s learning-efficiency advantage grow with a semantically organized base?

Experiment 2: the compiled channel (code delivered with this note). The qualitatively new capability: per-instance channel topology from dependency language. Programs are presented *scrambled*, each command carrying a linguistic dependency clause (“to begin, ...”; “after you reverse the order, double every value”), with distinct primitives per program so the reference is unambiguous. A pairwise scorer $s_{mj} = f(h_m, h_j)$ over frozen span representations predicts “ m directly follows j ” and is compiled into the inhibition matrix (ρ_{mj} lowered toward the invadable value where m follows j ; ρ_{jm} raised toward the extinguishing value), with a learned

entry scorer setting the initial condition from the “to begin” clause. Two supervision regimes: (a) *compiler-supervised* — edge labels from the generator (standing in for LLM-annotated dependency analyses; at real scale, prompt the base model for its own dependency parse) plus end-task loss; (b) *emergent* — end-task loss only. Metrics: task accuracy; edge AUC; itinerary-vs-true-order Kendall τ ; and *structural length generalization* — train on $L \leq 4$, test on $L = 5$, which the compiled channel should handle by construction (one more station in a topology it builds anyway) and which fixed-schedule models cannot represent.

Experiment 3: EPC A/B. Fine-tune a copy of the base with the error-predictive-coding objective, freeze, and rerun Experiments 1–2 against the standard base. Three measurable axes: probe score (linear accessibility), cap steps-to-threshold (trainability), held-out and compiled-channel transfer (semantic organization).

Experiment 4 (framing for later): the anti-CoT benchmark. Length-scaling curves — capped model vs. base-model chain-of-thought on state-manipulation programs of growing L — to substantiate S3, the regime claim. This requires a generative base configuration and is sequenced after 1–3.

4 Closing remark

The heteroclinic cap inverts the usual neuro-symbolic arrangement. Rather than a symbolic controller invoking a neural substrate, a neural compiler (the LLM) parameterizes a continuous dynamical runtime whose trajectory *is* the program: stations are subtasks, transitions are control flow, dwell is computation, and the trace is legible by construction. The sandbox established that the runtime works; the next phase tests whether the compiler’s semantics flow through it.